

- 1. Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

df = fetch_california_housing(as_frame=True).frame
df.hist(figsize=(10,8), bins=30)
plt.show()
df.plot(kind='box', subplots=True, layout=(3,3), figsize=(10,8))
plt.show()

print("Outliers detected per feature:")
for c in df.columns:
    Q1,Q3 = df[c].quantile([0.25,0.75])
    IQR = Q3-Q1
    out = df[(df[c] < Q1-1.5*IQR) | (df[c] > Q3+1.5*IQR)]
    print(c,":",len(out))
```

- 2. Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.**

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

df = fetch_california_housing(as_frame=True).frame
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
sns.pairplot(df.sample(300), diag_kind='kde')
plt.show()
```

- 3. Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.**

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

iris = load_iris()
X,y = iris.data, iris.target
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
```

```
plt.scatter(X_pca[:,0], X_pca[:,1], c=y, cmap='viridis')
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA: Iris 4D to 2D")
plt.grid()
plt.show()
print("Variance Ratio:", pca.explained_variance_ratio_)
```

-
- 4. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.**

```
import pandas as pd
import numpy as np

df = pd.read_csv("training_data.csv")
print("Training data:\n",df)
X = np.array(df.iloc[:, :-1])
y = np.array(df.iloc[:, -1])
h = X[y=='Yes'][0].copy()

for x in X[y=='Yes']:
    h[np.where(x!=h)]='?'
print("Final hypothesis:",list(h))
```

-
- 5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated.**

- a) Label the first 50 points $\{x_1, \dots, x_{50}\}$ as follows: if $(x_i \leq 0.5)$, then $x_i \in \text{Class1}$, else $x_i \in \text{Class1}$**
b) Classify the remaining points, $x_{51}, \dots, x_{100}$ using KNN. Perform this for $k=1,2,3,4,5,20,30$

```
import numpy as np
import matplotlib.pyplot as plt
from collections import Counter

np.random.seed(1)
x = np.random.rand(100)
tr_x, ts_x = x[:50], x[50:]
tr_y = ["Class1" if i<=0.5 else "Class2" for i in tr_x]

for k in [1,2,3,4,5,20,30]:
    preds=[]
    for p in ts_x:
        d = abs(tr_x-p)
        idx = d.argsort()[:k]
        lab = [tr_y[i] for i in idx]
        preds.append(Counter(lab).most_common(1)[0][0])

print(f'{k} Class1: {preds.count('Class1')} Class2: {preds.count('Class2')}')
```

```
plt.scatter(tr_x,[0]*50,c=['b' if l=='Class1' else 'r' for l in tr_y])
plt.scatter(ts_x,[1]*50,c=['b' if p=='Class1' else 'r' for p in preds], marker='x')
plt.show()
```

6. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

```
import numpy as np
import matplotlib.pyplot as plt

X = np.linspace(0,10,50)
y = np.sin(X)
X_mat = np.c_[np.ones(50),X]
def lwr(x,X,y,t):
    w = np.exp(-((X-x)**2).sum(axis=1)/(2*t*t))
    W = np.diag(w)
    theta = np.linalg.inv(X.T.dot(W).dot(X)).dot(X.T).dot(W).dot(y)
    return x.dot(theta)

x_test = np.linspace(0,10,100)
pred = [lwr(np.array([1,i]),X_mat,y,0.5) for i in x_test]
plt.scatter(X,y, color='red')
plt.plot(x_test,pred, color='blue')

plt.title('Locally Weighted Regression')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.grid()
plt.show()
plt.show()
```

7. Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error,r2_score

print("Demonstrating Linear Regression and Polynomial Regression\n")
X = fetch_california_housing().data[:,[2]]
y = fetch_california_housing().target
xtr,xte,ytr,yte = train_test_split(X,y,test_size=0.2,random_state=42)
m = LinearRegression().fit(xtr,ytr)
```

```

yp = m.predict(xte)

plt.scatter(xte,yte)
plt.plot(xte,yp,'r')
plt.show()
print("Linear Regression - California Housing Dataset")
print("Mean Squared Error:",mean_squared_error(yte,yp))
print("R^2 Score:",r2_score(yte,yp))
url="https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"
d=pd.read_csv(url,sep=r'\s+',names=["mpg","c","disp","hp","wt","acc","yr","org"],na_values="?").dropna()

X=d[['disp']]
y=d['mpg']
xtr,xte,ytr,yte=train_test_split(X,y,test_size=0.2,random_state=42)
p=PolynomialFeatures(2)
m=LinearRegression().fit(p.fit_transform(xtr),ytr)
yp=m.predict(p.transform(xte))

plt.scatter(xte,yte)
plt.scatter(xte,yp,c='r')
plt.show()
print("Polynomial Regression - Auto MPG Dataset")
print("Mean Squared Error:",mean_squared_error(yte,yp))
print("R^2 Score:",r2_score(yte,yp))

```

8. Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import *
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

d=load_breast_cancer()
xtr,xte,ytr,yte=train_test_split(d.data,d.target,test_size=0.2,random_state=42)
c=DecisionTreeClassifier().fit(xtr,ytr)
print("Accuracy =",accuracy_score(yte,c.predict(xte))*100)
print("Predicted Class :", "Benign" if c.predict([xte[0]])[0]==1 else "Malignant")

plt.figure(figsize=(12,8))
plot_tree(c,filled=True)
plt.show()

```

9. Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

```

from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split,cross_val_score

```

```

from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import *
import matplotlib.pyplot as plt

d=fetch_olivetti_faces(shuffle=True,random_state=42)
xtr,xte,ytr,yte=train_test_split(d.data,d.target,test_size=0.3,random_state=42)
g=GaussianNB().fit(xtr,ytr)
yp=g.predict(xte)

print("Accuracy:",accuracy_score(yte,yp)*100)
print(classification_report(yte,yp,zero_division=1))
print(confusion_matrix(yte,yp))
print("CV Accuracy:",cross_val_score(g,d.data,d.target,cv=5).mean()*100)
f,a=plt.subplots(3,5)

for x,i,t,p in zip(a.ravel(),xte,yte,yp):
    x.imshow(i.reshape(64,64),cmap='gray')
    x.set_title(str(t)+"/"+str(p))
    x.axis('off')
plt.show()

```

10. Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

```

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import *
import matplotlib.pyplot as plt

d = load_breast_cancer()
x = StandardScaler().fit_transform(d.data)
k = KMeans(n_clusters=2, random_state=42)
y = k.fit_predict(x)
print("Confusion Matrix:\n",confusion_matrix(d.target,y))
print("\nClassification Report:\n",classification_report(d.target,y))

p = PCA(2).fit_transform(x)
plt.scatter(p[:,0],p[:,1],c=y)
plt.scatter(*PCA(2).fit(x).transform(k.cluster_centers_).T,s=200,marker='X')
plt.title("K-Means Clustering")
plt.show()

```